TECHNICAL DOCUMENTATION

Terms of Reference Coverage

Wheat Water Productivity Dashboard

Ethiopian Institute of Agricultural Research · IWMI East Africa · FAO WaPOR Phase II
Generated 2026-07-29

ToR coverage

How this build maps to `ToR_Dashboard_Developer.docx`, and — as importantly — what it does not yet cover. Nothing below is inferred from the code; each row says where the work lives or why it is out of scope for a software build.

Objectives

#	OBJECTIVE	STATUS
1	Design frontend, backend, ML inference and integration layers	Done — the Architecture document
2	Interactive dashboard with multiple data-entry mechanisms	Done — admin unit, shapefile/GeoJSON upload, draw-on-map
3	Python analytical engine as a scalable backend service	Done — `backend/app/engine.py` behind `POST /api/analysis`
4	LightGBM deployed as a prediction service	Done — `model_service.py`, `/api/predict`, `/api/explain`
5	Scalable spatial database for inputs, artifacts and outputs	**Partial** — model artifacts are versioned on disk; upload registry and run store are in-memory. See below.
6	Integration into the EIAR web platform	Build-side done — relative-base bundle, sub-path safe, EIAR identity applied. Coordination with EIAR IT/GIS is an engagement activity.
7	System testing and quality assurance	Done — 102 automated checks across `tests/`
8	Technical documentation	Done — README, ARCHITECTURE, generated OpenAPI at `/docs`
9	Train EIAR staff	Out of scope for a software build — delivery activity

Work packages

WP1 — Inception, requirements, architecture. Architecture document produced. The approved `WWP Dashboard.html` prototype served as the click-through prototype and this build implements its user journeys; two visual decisions depart from it deliberately and are justified in the Architecture document § Visualization palette. Reviewing the live EIAR platform for integration constraints needs access to that platform and remains open.

WP2 — Frontend and dashboard. React 18 + Vite, Leaflet 1.9 for the map, charts as inline SVG. All three AOI journeys implemented. Validation runs on both sides: the client constrains selectors so an invalid combination is hard to express, and the server independently rejects unknown admin units, invalid season/system pairs, unknown years, degenerate polygons, non-EPSG:4326 shapefiles, boundaries outside Ethiopia, oversized extents, corrupt archives and unsupported file types — each with a message aimed at the user. EIAR colours, typography and logo mark are applied throughout, and the layout stacks cleanly from 1600 px to 420 px.

WP3 — Backend, API, analytical tool integration. Nine REST routes with generated OpenAPI documentation. Response caching (30-minute TTL, keyed on the canonical request) and `Idempotency-Key` support for retriable POSTs are both implemented and tested. The WaPOR data-access layer is abstracted behind `geodata.FeatureProvider`; the synthetic provider ships so the pipeline runs end-to-end, and swapping in live WaPOR retrieval touches one module.

WP4 — LightGBM integration and deployment. Versioned artifacts under `backend/models/vNNNN/` with `current.json` naming the active version. Inference accepts both a point and an area of interest. The retrain workflow scores the candidate and the active model on the same holdout and promotes only if the candidate does not degrade RMSE, so an update cannot silently make predictions worse and the previous version stays on disk to roll back to. Both explanation features the ToR asks for are present: a global feature-importance summary and a signed per-prediction breakdown from LightGBM's exact tree-path SHAP values.

WP5 — Website integration. The bundle builds with a relative base so it deploys under any sub-path behind a reverse proxy, and the backend can serve it directly. All five required supporting pages are written and reachable from the header and footer: methodology, data sources, user guide, how to cite, and disclaimer. **Web analytics (GA4) is not wired up** — it needs EIAR's property ID and a decision on consent handling, so it is left as a one-file addition rather than a guess.

WP6 — Documentation, training, support. Architecture, API and deployment documentation are in place. Training delivery and the three-month post-deployment support window are engagement activities.

What deployment still needs

Honest list of the gaps between this build and a production system on EIAR infrastructure:

1. **Live WaPOR provider.** Implement `assemble()` against FAO WaPOR v3 retrieval plus EthioSIS, SRTM and the survey layers, then set `geodata.PROVIDER`. Until then every number in the dashboard is synthetic — plausible and internally consistent, but not measured.
2. **Model trained on real observations.** The active model is trained on synthetic samples drawn from the documented generative process. Retrain on the Oromia and Afar field campaigns via `POST /api/model/retrain`.
3. **PostGIS for the spatial database.** Objective 5 is only partly met. The upload registry (`aoi._UPLOADS`) and completed-run store (`engine.RUNS`) are in-memory dictionaries, so a restart drops uploaded boundaries and CSV export links, and they cannot be shared across workers. Both are small, well-isolated modules to move to PostGIS tables, and the CSA boundary layer belongs there too.
4. **Redis for the cache** if the service runs more than one worker — `cache.TTLCache` is per-process, so with several workers the hit rate degrades and idempotency keys stop being reliable.

5. **GA4 analytics**, pending EIAR's property ID and consent approach.
6. **Amharic localization.** The language toggle is present but reports that the Amharic interface is planned; the UI strings are not yet externalized.
7. **Authentication**, if EIAR wants the retrain and model-management routes restricted. `POST /api/model/retrain` is currently unauthenticated and will replace the serving model on a successful upload — this should be protected before the service is publicly reachable.
8. **Map PNG export.** CSV export is implemented; the prototype's "Save map (PNG)" button is not, since a faithful map export needs server-side tile composition. The button was removed rather than left as a control that does nothing.

Test coverage

SUITE	CHECKS	COVERS
`tests/test_api_e2e.py`	47	All three AOI journeys, caching, idempotency, prediction, explanation, CSV export, and 11 validation/error paths
`tests/test_retrain.py`	4	Retrain, holdout comparison, promotion, continued serving
`tests/test_ui.mjs`	51	Rendering, chart integrity, label overflow, table views, results-panel open/close and map re-sync, pixel inspect, both polygon-finish paths, content pages, four viewport widths, console cleanliness

All 102 pass against the current build, verified from a clean state (model directory deleted, so the run also covers first-boot training).